

AutoIRT Replication — Progress Report 2: DET-Phase Calibration

Prepared by: Shalin Bhavsar
Supervisor: Dr. Yi Zheng, Arizona State University

Reporting period: July 19 – July 26, 2026

Overview

This report covers three things from your last round of feedback: how warm-start and cold-start are actually implemented (with citations to where the paper defines them), what I found when I looked into the MCEM termination rule, and results from the DET-style two-item-type calibration (Phase 2).

1 Warm-start / cold-start / jump-start implementation

The paper introduces these as three general calibration scenarios before it gets into the DET-specific analysis (p. 3, “Online English Proficiency Testing” — “There are three primary use cases for calibration”):

- **Cold-start:** new pilot items with no response data at all, calibrated from item features alone, while the operational bank is re-calibrated in parallel.
- **Jump-start:** a limited number of responses have come in for new pilot items during piloting, on top of the operational bank.
- **Warm-start:** re-calibrating an existing operational bank to reflect recent data, e.g. after a UI change or a shift in the test-taker population.

Same page also gives the per-session item counts I used: 18 Y/N Vocab items and 9 ViC items per session.

The offline DET analysis (p. 6, “Offline calibration analysis for DET”) turns these into concrete train/test splits, and this is what `run_det_experiment.py` implements directly:

1. **Cold-start:** pick a split date (2024-05-22 in the paper), randomly hold out 50% of items as pilot items. Train on operational-item responses before the split date. Test on everything after the split date, pilot and operational both.
2. **Jump-start ($R = 20, 40, 80$):** same item/date split, but training also gets the first R post-split responses per pilot item. Test is whatever’s left over.
3. **Warm-start:** no item holdout, just a date split. The paper uses two dates, 2024-05-08 and 2024-05-15.

I don’t have real DET timestamps, so each simulated session gets a random “day” over a 79-day window standing in for the paper’s real window (2024-04-01 to 2024-06-18), and the split days are computed from their actual dates rather than picked arbitrarily — day 51 for cold/jump-start, days 37 and 44 for the two warm-start splits.

- `split_cold_start()` masks training rows to operational items with `day < split_day`; test is everything with `day >= split_day`.
- `split_jump_start()` does the same, then walks through each pilot item’s post-split responses in order (day, then a response counter to break same-day ties) and keeps the first R per item for training.
- `split_warm_start()` is just a date mask, no item logic.

Both item types get their own ability parameter, matching the paper’s Method section (p. 3: “We separately model the ViC and Y/N Vocab item parameters and ability, so that each item type has its own associated θ_s ”), and their own fixed chance parameter from the offline analysis section (p. 6): $c = 0.25$ for Y/N Vocab since it’s multiple choice, $c = 0$ for ViC since guessing isn’t realistic there.

2 MCEM termination rule

Algorithm 1 (top of p. 4) really is just **for each EM iteration do** — no stopping condition in it anywhere. The paper’s choice of 4 iterations comes from a separate empirical note later in the Results section (training loss was roughly flat within 4 steps for their simulation settings), not a general rule.

The standard reference for an automated MCEM stopping rule is Booth & Hobert (1999, JRSS B, 61(1), 265–285) — they build a confidence interval around each iteration’s parameter update using its Monte Carlo error, and stop once the update is statistically indistinguishable from zero. Their full method needs machinery I don’t have here (standard errors on each item parameter from repeated draws), so I built a simpler version in the same spirit: track the training loss I’m already logging each step, and stop once its relative change has stayed under 0.5% for most of a short window of recent steps, with a hard cap so it can’t run forever.

Re-running the original 400-item / 20,000-session scenario with this in place, every run needed well over twice the paper’s 4 iterations to settle — consistently into double digits, never close to 4. Running to real convergence improved warm-start test loss from 0.583 to 0.535 and tightened the spread across seeds. Cold-start barely moved, which tells me the step count was never what was holding cold-start back — that lines up with the item-bank-size effect from the last report.

One thing I want to flag rather than gloss over: no simple threshold rule handles every trajectory cleanly. While testing this on the Phase 2 runs below, one condition (ViC Jump 20) converged fine under an earlier, stricter version of this rule, then failed to converge once I widened the rule to fix two other conditions’ noisy trajectories. Fixing one failure mode introduced a different one elsewhere. Instead of continuing to tune thresholds against a handful of runs, I’m treating `hit_max_em_steps` as a legitimate outcome for the noisiest, smallest-data conditions, and flagging it plainly when it happens rather than hiding it (see the Jump 20 row below).

3 Phase 2 results: DET-style two-item-type calibration

Item banks: 150 Y/N Vocab items, 80 ViC items (the real DET has 3,290 and 585 — mine are scaled down so iteration is fast enough to debug), 1,500 simulated sessions, 50% of each bank held out as pilot items.

Y/N Vocab

Split	Loss	Pearson	Spearman	EM steps	Paper's AutoIRT (Loss / Pears. / Spear.)
Cold	0.594	0.519	0.404	9	0.456 / 0.785 / 0.822
Jump 20	0.544	0.836	0.697	15	0.428 / 0.936 / 0.944
Jump 40	0.529	0.857	0.736	13	0.426 / 0.955 / 0.962
Jump 80	0.523	0.937	0.802	16	0.423 / 0.971 / 0.973
Warm 05-08	0.528	0.925	0.780	8	0.431 / 0.987 / 0.991
Warm 05-15	0.528	0.923	0.788	9	0.423 / 0.987 / 0.989

ViC

Split	Loss	Pearson	Spearman	EM steps	Paper's AutoIRT (Loss / Pears. / Spear.)
Cold	0.556	0.523	0.432	19	0.435 / 0.742 / 0.786
Jump 20	0.489	0.857	0.808	20*	0.381 / 0.954 / 0.959
Jump 40	0.474	0.880	0.882	18	0.374 / 0.978 / 0.979
Jump 80	0.485	0.915	0.860	18	0.371 / 0.989 / 0.989
Warm 05-08	0.475	0.928	0.910	18	0.359 / 0.998 / 0.997
Warm 05-15	0.463	0.930	0.913	17	0.355 / 0.998 / 0.997

*Hit the max EM step cap without fully converging; see Section 2.

(Paper numbers are Table 1, p. 7 for Y/N Vocab and Table 2, p. 7 for ViC, AutoIRT rows only.)

The ordering matches the paper exactly for both item types — Cold worst, Jump-start climbing with R, Warm-start best. Everything is quantitatively below the paper's numbers though, most sharply at Cold-start (Y/N Vocab 0.519 vs. their 0.785; ViC 0.523 vs. their 0.742). I don't think that's a separate problem from the item-bank-size finding in the last report — the operational banks here are small after the 50% pilot holdout (75 Y/N Vocab items, 40 ViC items), and ViC, with the smaller bank of the two, comes out weaker on cold-start than Y/N Vocab, which is exactly what that earlier finding would predict.

Next steps

- Scale up the item bank sizes here and see if the gap narrows the same way it did in the earlier sweep.
- Move on to the AutoGluon comparison I mentioned last time, to see how much of the remaining warm-start gap comes from the hand-built ensemble versus their AutoML backend.